

Landform Mesh Tiling Server

Contents

Introduction	2
AWS Setup	2
1. S3 Bucket Setup	3
2. Create Elastic Beanstalk Environment for Master Server	4
3. Configure DNS	5
4. Configure HTTPS	6
1. Generate Certificate Signing Request	6
2. Generate Signed Certificate	7
3. Install Signed Certificate	7
5. Restrict to JPL IPs	8
6. Configure Elastic Beanstalk Environment	9
7. Deploy Server Binaries to Elastic Beanstalk	9
8. Deploy Worker Binaries to EC2 Auto Scale Group	10
1. Setup Security	10
2. Create Launch Template	10
3. Upload New Worker	11
4. Create Auto Scaling Group	12
5. To Remote in to an EC2 Instance in the Autoscale Group	12
REST API Manual	13
Create Project: POST /api/projects/ <i>name</i>	13
Upload Data: POST /api/projects/ <i>name</i> /upload	14
Run Project: POST /api/projects/ <i>name</i> /run	16
Get Project Metadata: GET /api/projects/ <i>name</i>	17
List Projects: GET /api/projects	18
Get Project Result URL: GET /api/projects/ <i>name</i> /result	19
View Project: GET /api/projects/ <i>name</i> /view	19
Delete Project: DELETE /api/projects/ <i>name</i>	20
Task API	21
Get Master Task ID: GET /api/tasks/master/id	22
Get Task Metadata: GET /api/tasks/ <i>id</i>	23
Get Task Log: GET /api/tasks/ <i>id</i> /log	23

Test Procedures	24
Authentication Test	24
REST API Tests	25
Setup	25
Test Sequence	26

Introduction

The Landform Mesh Tiling Server securely converts textured triangle meshes to tilesets in the 3DTiles format:

<https://github.com/AnalyticalGraphicsInc/3d-tiles>

Potentially large meshes are broken up into a hierarchy of tiles that can be scalably streamed to clients.

This software also supports serving the generated 3DTiles tilesets to clients, and so is also referred to as a “Geometry Streaming Server”.

Included in this manual are:

- instructions for deploying to Amazon Web Services
- a manual for the REST API, which is the interface for uploading input meshes and converting them to 3DTiles tilesets
- test instructions.

AWS Setup

The AWS services used are

- Elastic Compute Cloud EC2 (<https://aws.amazon.com/ec2>)
- Simple Cloud Storage Service S3 (<https://aws.amazon.com/free/storage>)
- DynamoDB (<https://aws.amazon.com/dynamodb>)
- Simple Queue Service SQS (<https://aws.amazon.com/sqs>)
- Elastic Beanstalk EB (<https://aws.amazon.com/elasticbeanstalk>)
- Identity & Access Management IAM (<https://aws.amazon.com/iam>)
- Managed Cloud DNS Route 53 (<https://aws.amazon.com/route53>) (optional)
- Certificate Manager (<https://aws.amazon.com/certificate-manager>)

A full deployment includes

1. a master server running on an EC2 instance managed with Elastic Beanstalk
2. a group of workers running on one or more EC2 instances managed as an EC2 Auto Scaling Group.

The master server includes

- a web interface that can be securely accessed by end users

- a REST API that can be used to securely integrate with other systems
- a backend task which orchestrates tiling workflows.

The workers perform tasks defined by the master and are not externally accessible.

Each deployment on AWS is configured with a unique venue name used to partition the DynamoDB, S3, and SQS entries specific to that venue.

1. S3 Bucket Setup

Input and output datasets are stored in an AWS S3 bucket. The bucket needs to be fully accessible by the account in which the server components (master and workers) are deployed. Typically this is ensured by using the same account to create the S3 bucket as for deploying the server components.

The bucket typically also needs external `https` access so that results can be accessed for viewing and downstream use. The instructions below show how to set this up limited to clients with JPL IP addresses.

1. `http://goto.jpl.nasa.gov/awsconsole`
2. Log in with the same AWS account you use to deploy the server components
3. Select region `us-west-1` (North California)
4. Services -> Storage -> S3
5. Create Bucket
 1. enter a bucket name - note it must be globally unique across all S3 bucket names
 2. Create
6. Select bucket in list
 1. Permissions -> Bucket Policy
 1. paste the following, replacing `BUCKET_NAME` with your bucket name

```
{
  "Version": "2012-10-17",
  "Id": "S3PolicyId1",
  "Statement": [
    {
      "Sid": "allow readonly from JPL",
      "Effect": "Allow",
      "Principal": {
        "AWS": "*"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::BUCKET_NAME/*",
      "Condition": {
        "IpAddress": {
          "aws:SourceIp": [
            "128.149.0.0/16",
            "137.78.0.0/16",
```

```

        "137.79.0.0/16",
        "137.228.0.0/16"
    ]
}
}
}
]
}
2. Save
2. Properties -> Static website hosting
    1. Use this bucket to host a website
    2. Index document: index.html
    3. Save

```

2. Create Elastic Beanstalk Environment for Master Server

This step creates the Elastic Beanstalk application and environment into which the master server will be deployed.

1. Log in to the AWS web console with an AWS account and region that can access the S3 bucket you setup above.
 2. Services -> Compute -> Elastic Beanstalk
 3. Create application and assign it a unique name
 4. Create new web sever environment
 1. assign a unique environment name
 2. Platform: **docker**
 3. Configure more options
 1. Modify instances
 1. Instance type: **t2.medium**
 2. Instance security groups: default creates a security group that can only talk to the load balancer which is what we want
 4. Modify Capacity -> Load balanced, min = 1, max = 1
 5. Modify security -> IAM instance profile: typically set this to match your AWS account
 6. Modify network
 1. Visibility: Public
 2. Load balancer subnets:


```
us-west-1c subnet-148d7971 172.31.16.0/20
```
 3. Instance subnets: same as load balancer
5. Create Environment - it takes a few minutes
6. Adjust elastic load balancer settings.
 1. Log in to the AWS web console with the same AWS account and region as above.
 2. Services -> Compute -> EC2
 3. Load Balancing -> Load Balancers
 4. Select the load balancer corresponding to the elastic beanstalk envi-

- ronment - on the “Instances” tab you should see an instance with the same name as the elastic beanstalk environment
5. On the “Description” tab for the load balancer, click “Edit idle timeout” and set it to 1800 seconds
 6. On the “Listeners” tab, click “Edit”, and then change cipher.
 1. Choose “Custom Security Policy”
 2. under SSL Protocols make sure TLSv1 is unchecked (TLSv1.1 and greater are OK)
 3. under SSL Ciphers make sure only ones from the following NASA approved list https://jplsoc2.jpl.nasa.gov/jplsoc/compliance/ciphers/supported_ciphers.txt are checked (the TLS_ prefix may be missing)
 - TLS_DHE_RSA_WITH_AES_128_GCM_SHA256
 - TLS_DHE_RSA_WITH_AES_256_GCM_SHA384
 - TLS_DHE_DSS_WITH_AES_128_CBC_SHA
 - TLS_DHE_DSS_WITH_AES_256_CBC_SHA
 - TLS_DHE_DSS_WITH_AES_128_GCM_SHA256
 - TLS_DHE_DSS_WITH_AES_256_CBC_SHA256
 - TLS_DHE_DSS_WITH_AES_256_GCM_SHA384
 - TLS_DHE_DSS_WITH_AES_128_CBC_SHA256
 - TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA
 - TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256
 - TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256
 - TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA384
 - TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
 - TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA
 - TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256
 - TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA
 - TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA384
 - TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA256
 - TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA
 - TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384
 4. save the changes
 5. once you complete the DNS setup steps below, for deployments at JPL you can check that you got them all by going to http://jplsoc2.jpl.nasa.gov/ciphers/ciphers_check.cfm and verifying that no ciphers are shown in red for the DNS name of your depolyment.

3. Configure DNS

This step is optional. It configures a public DNS entry to redirect to the Elastic Beanstalk environment configured above. These instructions use the Amazon Route 53 DNS service, but any DNS provider that supports CNAME records should work.

If you forgo this step you can still access the server at a URL like `https://EBENV.AWSREGION.elasticbeanstalk.com` where `EBENV` is the Elastic Beanstalk environment name and `AWSREGION` is the AWS region containing it.

1. Log in to the AWS web console, does not need to be the same AWS account and region as above.
2. Services -> Networking -> Route 53
3. select hosted zone
4. Create record set
 1. Name: DNS name you want end-users to use to access the server
 2. Type: `CNAME`
 3. TTL: 60
 4. Value: `EBENV.AWSREGION.elasticbeanstalk.com` - this must match the Elastic Beanstalk environment created above
 5. Routing policy: `simple`
 6. Create

4. Configure HTTPS

This step uses the JPL and AWS certificate managers to generate and deploy cryptographic certificates to enable secure HTTPS connection to the server.

The overall flow is

1. generate an RSA private key
2. use the key to sign a certificate signing request (CSR)
3. submit the CSR to the JPL certificate manager to generate an HTTPS certificate (PEM)
4. register the certificate and key to the AWS certificate manager
5. associate the certificate with an Elastic Beanstalk load balancer listener which will proxy HTTPS communication with the Landform master server.

Many steps below require the end-user DNS name for your server which will be noted as `DNS_NAME`.

1. Generate Certificate Signing Request

You will need a command line that includes the `openssl` tool. On Windows one option is Git bash (<https://gitforwindows.org>).

1. generate an RSA private key: `openssl genrsa > DNS_NAME.key`
2. generate CSR: `openssl req -new -key DNS_NAME.key -out DNS_NAME.csr`
 - Common name: `DNS_NAME`
 - other values will be specific to your deployment

2. Generate Signed Certificate

Now you will submit the CSR to a certificate signing authority to generate a signed certificate. The instructions below can be used within JPL.

1. Login to JPL Certificate manager at <https://ssl.jpl.nasa.gov>
2. Manage my certificates -> Request a Certificate
 1. LDAP group: select an LDAP group for managing the certificate
 2. Machine name: `DNS_NAME`
 3. Server type: Apache
 4. CSR: paste entire contents of the CSR here
 5. wait a few minutes, you should get an email when the cert is ready
3. Manage my certificates
 1. click on `DNS_NAME`
 2. paste entire contents of certificate to `DNS_NAME.pem`. It should have a form like this
CERTIFICATE:
-----BEGIN CERTIFICATE-----
(cert)
-----END CERTIFICATE-----
INTERMEDIATE CERTIFICATE:
-----BEGIN CERTIFICATE-----
(intermediate cert)
-----END CERTIFICATE-----
ROOT CERTIFICATE:
-----BEGIN CERTIFICATE-----
(root cert)
-----END CERTIFICATE-----

3. Install Signed Certificate

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Security -> Certificate Manager -> Import a certificate
 1. Paste
-----BEGIN CERTIFICATE-----
(cert)
-----END CERTIFICATE-----
from your PEM file into the certificate body field.
 2. Paste
-----BEGIN CERTIFICATE-----
(intermediate cert)
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
(root cert)
-----END CERTIFICATE-----
from your PEM file into the certificate chain field.

3. Paste entire contents of your RSA private key file into the certificate private key field.
4. Review and Import -> Import
5. Open accordion for the cert
 - edit name tag to be your Landform master server DNS name
 - Details -> Identifier GUID
 - Make a note of the GUID
3. Services -> Compute -> Elastic Beanstalk
 1. Navigate into the Elastic Beanstalk application and environment you configured above
 2. Configuration -> Modify load balancer
 1. Classic load balancer
 2. Turn off existing listener
 3. Add listener
 - Listener port: 443
 - Listener protocol: HTTPS
 - Instance port: 80
 - Instance protocol: HTTP
 - Select the SSL certificate ID. These may appear as `*.jpl.nasa.gov`, in which case you need to find the GUID matching the cert in the AWS Certificate Manager.
 4. Apply

5. Restrict to JPL IPs

This step is optional but recommended for deployments within JPL. It restricts access to your Landform master server to clients within the JPL IP address space.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Configuration -> Instance settings, note down the EC2 security group
5. Services -> Compute -> EC2 -> Security Groups
 1. Search for the instance security group noted above
 2. Look at the security group referenced in its inbound source. This is the ELB security group.
 3. Search for and select the ELB security group and change its inbound settings to:
 - HTTPS TCP 443 128.149.0.0/16
 - HTTPS TCP 443 137.78.0.0/16
 - HTTPS TCP 443 137.79.0.0/16
 - HTTPS TCP 443 137.228.0.0/16

6. Configure Elastic Beanstalk Environment

This step configures the Elastic Beanstalk environment with specifics of your deployment.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Software -> Environment variables
 - **NODE_ENV**: **production** (**integration** for testing)
 - **SESSION_SECRET**: any private string
 - **TOKEN_SECRET**: any private string
 - **SAML_ENTRY_POINT**: for depolyments within JPL you will need to contact JPL IT to set up SSO for your deployment and typically this field will have a form like
`https://SSO_HOST.jpl.nasa.gov/oamfed/idp/initiatesso?providerid=https://DNS_NAME`
where **SSO_HOST** is **ssoint** for integration testing and **sso1** for production, and **DNS_NAME** is your server DNS name.
 - **SAML_CERT**: for deployments within JPL typically copy the X509Certificate field from `https://SSO_HOST.jpl.nasa.gov/oamfed/idp/metadata`.
 - **LANDFORM_AWS_REGION**: same region you used to setup the Elastic Beanstalk environment
 - **LANDFORM_AWS_PROFILE**: **default**
 - **LANDFORM_VENUE**: choose a venue name for the deployment; it must match the venue name in the worker configuration below
 - **LANDFORM_S3_URL**: **s3://BUCKET_NAME** where **BUCKET_NAME** is the S3 bucket you setup above
 - **LANDFORM_LDAP_GROUP**: LDAP group for SSO authentication
5. Apply

7. Deploy Server Binaries to Elastic Beanstalk

The following instructions assume you have a `landform-master[-VERSION].zip` release bundle.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> Elastic Beanstalk
3. Navigate into the Elastic Beanstalk application and environment you configured above
4. Upload and Deploy
5. Choose File: `landform-master[-VERSION].zip`
6. Version Label: **VERSION**
7. Deploy - it takes a few minutes

Once the server is deployed it will be live. To shut it down, either terminate

the corresponding Elastic Beanstalk environment or set its autoscale group to max 0 instances. The latter may be more convenient because terminating the environment seems to lose its configuration. One way to reduce the autoscale group to 0 instances is to use the Python `eb` command line tool. Using Python 3:

```
pip install awsebcli
eb scale 0 ENVIRONMENT --profile=PROFILE
```

where `ENVIRONMENT` is the Elastic Beanstalk environment name and `PROFILE` is the AWS profile to use.

8. Deploy Worker Binaries to EC2 Auto Scale Group

The following instructions assume you have a `landform-worker[-VERSION].zip` release bundle and an `ec2userdata.txt` file.

1. Setup Security

These steps only need to be performed once before your first deployment.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Network & Security -> Security Groups (optional - only if you don't already have a security group that enables RDP and you want to have remote access to the EC2 instances in the autoscale group for debugging or maintenance)
 1. Create Security Group
 2. Name: `RDP Only` (recommended, but can be any name)
 3. Description: `RDP Only` (recommended, but can be any name)
 4. Rules
 1. Add Rule
 - Type: `RDP`
 - Source:
`Custom: 128.149.0.0/16, 137.78.0.0/16, 137.79.0.0/16, 137.228.0.0/16`
this will restrict RDP access to JPL IP addresses
4. Services -> Compute -> EC2
5. Network & Security -> Key Pairs (optional - only if you want to be able to log in to the EC2 instances in the autoscale group for debugging or maintenance)
 1. you can use a tool like `ssh-keygen -t rsa` to generate a key pair

2. Create Launch Template

These instructions only need to be run once for a new venue or when the venue configuration (`ec2userdata.txt`) changes.

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Instances -> Launch Templates
4. Create New Launch Template
 - If you have already created a template for a venue then you can
 - select “Create a new template version”
 - select the previous template in “Launch template name”
 - select the most recent version of the previous template in “Source Template Version”
 - the new template will be pre-populated with values from the old template
 - most likely the only field you’ll need to replace is “User Data”
 - after saving the new version
 - * select the template in the list
 - * Actions -> Set default version
 - * select the newest version
 - * click set as default version
 - Launch Template Name: select your launch template
 - AMI ID: `ami-0df605282263fb1c9` (Microsoft Windows Server 2016 Base 64-bit)
 - Instance Type: `t2.2xlarge` recommended, other instance types (<https://aws.amazon.com/ec2/instance-types>) can be chosen for different price (<https://aws.amazon.com/ec2/pricing/on-demand>) and performance tradeoffs
 - Key Pair: the key pair you selected above
 - Network Type: `classic`
 - Availability Zone: same AWS region that you used to set up the Elastic Beanstalk environment
 - Security Groups: `RDP Only` (or whatever security group you selected above)
 - Tags -> Add Tag
 - Key: `Name` - note this must be capitalized exactly as shown
 - Value: assign a name to will identify the EC2 instances in the group
 - Advanced
 - IAM Instance Profile: typically set this to match your AWS account
 - User Data: cut and paste the entire contents of `ec2userdata.txt`
5. Create Launch Template

3. Upload New Worker

These instructions only needs to be run when the Landform worker version changes (`landform-worker[-VERSION].zip`).

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Storage -> S3
3. select your S3 bucket
4. select or create a subfolder matching your venue name
5. select or create subfolder **app**
6. if `landform-worker[-VERSION].zip` exists, delete it
7. upload `landform-worker[-VERSION].zip`
8. right click on `landform-worker[-VERSION].zip` and rename to `landform-worker.zip`
9. you will need to restart all EC2 instances in the autoscale group to pick up the changes; one way to do that is to delete any existing auto scaling group and then re-create following the instructions below.

4. Create Auto Scaling Group

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Auto Scaling -> Auto Scaling Groups -> Create Auto Scaling Group
4. Launch Template
5. select the launch template you created above
6. Next Step
 - Group Name: assign a group name, e.g. same as launch template name
 - Subnet: `172.31.16.0/20` | Default in `AWS_REGION`
7. Configure Scaling Policies
 - Use scaling policies to adjust the capacity of the group
 - Scale Between: 1 and 4 instances (recommended, but other parameters can be chosen for different price/performance tradeoffs)
 - Metric type: **Average CPU Utilization**
 - Target value: 60
 - Instances need: 300 seconds to warm up after scaling
8. Review
9. Create Auto Scaling Group

5. To Remote in to an EC2 Instance in the Autoscale Group

1. Log in to the AWS web console with the same AWS account and region that you used to set up the Elastic Beanstalk environment.
2. Services -> Compute -> EC2
3. Instances -> Instances
4. Right click on an instance in the group
5. if this is your first time connecting

1. Get Password
2. choose file and select the *private* key from the key pair you specified when creating the launch template above
3. Decrypt Password
4. copy to clipboard
6. Download remote desktop (RDP) file
7. Double click RDP file to open remote desktop
8. Username: **admin**, password as above
9. The server should be located in **C:\tileserver**. You can tail the server log by running the PowerShell command


```
Get-Content c:\tileserver\log\log-tilingserver-startworker*.txt -Wait -Tail 30
```
10. The EC2Launch (<https://docs.aws.amazon.com/AWSEC2/latest/WindowsGuide/ec2-windows-user-data.html>) log for the user data script should be at


```
C:\ProgramData\Amazon\EC2-Windows\Launch\Log\UserdataExecution
```

You may need to show hidden files and folders (<https://support.microsoft.com/en-us/help/14201/windows-show-hidden-files>) to see **C:\ProgramData**.

REST API Manual

All API methods require an API token to be set as an **x-landform-token** HTTP header or **landform-token** cookie.

Unless otherwise specified, all API arguments may be specified as either URL query parameters or as fields in a **application/json** or **application/x-www-form-urlencoded** HTTP request body. If both the query parameter and body field are present the former takes precedence.

Create Project: POST **/api/projects/*name***

Create the named project. Implements the task API.

Accepts the following arguments:

- *tilingscheme*: tiling scheme; one of **Bin**, **QuadX**, **QuadY**, **QuadZ**, **Oct**, or **UserDefined**; default **Bin**
- *skirtmode*: skirt mode; one of **X**, **Y**, **Z**, **None**, **Normal**; default **None**
- *reconmethod*: reconstruction method; one of **Poisson**, **FSSR**; default **Poisson**
- *facespertile*: target maximum faces per tile; default 2000
- *tileresolution*: maximum image resolution per tile; default 256
- *projecttype*: project type; one of **GenericTiling** or **MSL**, default **GenericTiling**.

- *exportmeshformat*: additional mesh format to write, one of `obj`, `ply`, `stl`, or none; default none
- *exportimageformat*: additional image format to write, one of `tif`, `png`, `jpg`, or none; default none
- *maxleafgroupsize*: maximum number of leaves to process as a group; default 32

Fails with HTTP status 400 (bad request) if a project with the same name already exists.

If multiple calls to this API are made in rapid succession with the same project name only one will succeed. However, in this case it is possible that the HTTP status of more than one of the calls may be HTTP 200 (OK).

The return status may be HTTP 200 even if one or more arguments are invalid. However the project will not actually be created in that case.

Example: create project “testproj”

Request

```
POST /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=-----WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8

{
  "id": 1,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537227661847,
  "ended": 1537227672413
}
```

Upload Data: POST /api/projects/*name*/upload

Upload data for the named project. Implements the task API.

Accepts the following arguments in a `multipart/form-data` encoded HTTP request body:

- *mesh*: mesh data file (required); data format will be implied from filename extension
- *texture*: texture image file (optional); data format will be implied from filename extension
- *tileid*: tile ID string; must be given if and only if the tiling scheme for the project is **UserDefined** (this argument may also be specified as a URL parameter).

Fails with HTTP status 400 (bad request) if the named project does not exist.

If the project is deleted or run quickly after a call to this API then it is possible that the HTTP status will be 200 (OK) but the upload will not ultimately succeed.

Data formats are implied from the filename extension.

Supported extensions for mesh files include `.obj` and `.ply`. PLY files may be ASCII or binary.

Supported extensions for image files include `.tiff`, `.tif`, `.jpg`, and `.png`. TIFF files may be 8 or 16 bit. Images may be greyscale or 3 band color.

If more than upload is made with the same mesh basename (filename excluding extension) in a project then only the most recently uploaded data will be used.

All uploads must be complete before a project starts running. The inputs used by a project are determined at the time the project is first run. Additional inputs uploaded to the project after that time are ignored, even if the project is re-run.

Input data is validated when a project is run, not on upload.

Example: upload “inputMeshSmall.ply” and “inputImage.png” to “testproj”

Request

```
POST https://landform.hi.jpl.nasa.gov/api/projects/testproj/upload HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW
```

```
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="mesh"; filename="inputMeshSmall.ply"
Content-Type:
```

```
DATA
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="texture"; filename="inputImage.png"
Content-Type: image/png
```

```
DATA
-----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 2,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537228976377,
  "ended": 1537228980192
}
```

Run Project: POST /api/projects/*name*/run

Initiate a run of a project. Implements the task API.

Fails with HTTP status 400 (bad request) if the named project does not exist.

If the project is deleted quickly after a call to this API then it is possible that the HTTP status will be 200 (OK) but the run will fail.

This task only initiates the execution of a project. To determine whether the project execution has completed, do one of the following:

- poll the project metadata via /api/projects/**name** and wait for `Project.FinishedRunning=true`
- get the project result URL via /api/projects/**name**/result?redirect=false and poll it until the status is HTTP 200 (OK).

It is safe to issue this command more than once. However, project results are only computed once: in the absence of errors, subsequent runs of a project after the first run has begun have no effect.

Example: run project “testproj”

Request

```
POST /api/projects/testproj/run HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary7MA4YWxkTrZu0gW
```

```
-----WebKitFormBoundary7MA4YWxkTrZu0gW--
```

Response

```
HTTP/1.1 200
```

```

status: 200
content-type: application/json; charset=utf-8

{
  "id": 3,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537229779144,
  "ended": 1537229780231
}

```

Get Project Metadata: GET /api/projects/*name*

Get JSON metadata for a project.

Fails with HTTP status 400 (bad request) if the named project does not exist.

The project metadata is returned as a JSON object with at least the following fields:

- **Project**
 - **Name**: project name
 - **TilingScheme**, **SkirtMode**, **FacesPerTile**, **TileResolution**: correspond to the options when the object was created
 - **StartedRunning**: whether the project has started execution
 - **FinishedRunning**: whether the project has finished execution
- **Inputs**: array of project inputs
 - **Name**: name of this input
 - **MeshUrl**: URL of the mesh for this input
 - **ImageUrl**: URL of the image for this input
 - **Processed**: whether this input has been processed yet
 - **ImageBands**, **ImageWidth**, **ImageHeight**: image metadata, null if the input has not yet been processed
- **NumNodes**: the total number of 3DTiles hierarchy nodes defined for the project, or null if that has not been computed yet
- **NumProcesedNodes**: number of hierarchy nodes processed so far, or null if the project has not yet begun execution
- **OutputUrl**: the URL at which the final `tileset.json` is expected, see `/api/projects/*name*/result` for more info

Example: get metadata for project “testproj”

Request

```

GET /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN

```

Response

HTTP/1.1 200

status: 200

content-type: application/json; charset=utf-8

```
{
  "Project": {
    "Name": "testproj",
    "TilingScheme": "Bin",
    "SkirtMode": "None",
    "ReconMethod": "Poisson",
    "FacesPerTile": 2000,
    "TileResolution": 256,
    "TilesDefined": true,
    "ProjectType": "GenericTiling",
    "StartedRunning": true,
    "FinishedRunning": true
  },
  "Inputs": [
    {
      "Name": "inputMeshSmall",
      "MeshUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/input/testproj/inputMes",
      "ImageUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/input/testproj/inputMe",
      "Processed": true,
      "ImageBands": 3,
      "ImageWidth": 4096,
      "ImageHeight": 4096
    }
  ],
  "NumNodes": 111,
  "NumProcessedNodes": 111,
  "OutputUrl": "https://landlords-dev.s3.amazonaws.com/landformweb/www/testproj/tileset.json"
}
```

List Projects: GET /api/projects

Get a list of the existing project names.

The project names are returned as a JSON array of strings.

Example:

Request

GET /api/projects HTTP/1.1

Host: https://landform.hi.jpl.nasa.gov

x-landform-token: API_TOKEN

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
[
  "testproj",
  "testproj1"
]
```

Get Project Result URL: GET /api/projects/*name*/result

Fetch 3DTiles (<https://github.com/AnalyticalGraphicsInc/3d-tiles>) `tileset.json` result for a project.

By default this API will return a HTTP 302 (found) redirect to the `tileset.json` file stored on AWS. If execution of the project is not yet completed then AWS will return HTTP 403 (forbidden).

If the argument `redirect=false` is specified then the output is the URL to the tileset with no redirect.

Example: get result URL for project “testproj” without redirect

Request

```
GET /api/projects/testproj/result?redirect=false
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
https://landlords-dev.s3.amazonaws.com/landformweb/www/testproj/tileset.json
```

View Project: GET /api/projects/*name*/view

Launch a web-based 3D viewer for a completed project.

By default this API will return a HTTP 302 (found) redirect to a web-based viewer for the project.

If the argument `redirect=false` is specified then the output is the viewer URL with no redirect.

Example: launch 3D viewer for project “testproj”

Request

```
GET /api/projects/testproj/view HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 302 Found
location: /viewer/index.html?TilesetURL=https%3A%2F%2Flandlords-dev.s3.amazonaws.com%2Flandf
content-type: text/html; charset=utf-8
```

Delete Project: DELETE /api/projects/*name*

Delete the named project. Implements the task API.

Fails with HTTP status 400 (bad request) if the named project does not exist or is currently running.

If multiple calls to this API are made in rapid succession with the same project name only one will succeed. However, in this case it is possible that the HTTP status of more than one of the calls may be HTTP 200 (OK).

If the project is run quickly after this API is called then the deletion may fail even though the status of the delete call may have been HTTP 200.

Example: delete project “testproj”

Request

```
DELETE /api/projects/testproj HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 1,
  "running": false,
  "success": true,
  "exitCode": 0,
  "error": null,
  "started": 1537227661847,
  "ended": 1537227672413
}
```

Task API

Many API calls launch a task. By default such calls are synchronous in that they will not return a response until the task completes. At that point an `application/json` response will normally be sent with the following task metadata:

```
{
  id: server assigned unique task identifier
  name: server assigned task name (may not be unique)
  running: false
  success: whether task succeeded (true or false)
  exitCode: 0 on success, nonzero integer or string error code on failure
  error: error message if any
  started: task start time (ms since epoch on server)
  ended: task end time (ms since epoch on server)
}
```

The HTTP status will be 200 (ok) if the task completed successfully or 500 (server error) if it failed.

If the server encountered an error processing the request (e.g. invalid API call) or failed to launch the task it will return an `application/json` failure response containing only the following error metadata:

```
{
  success: false
  error: error message if any
}
```

In this case the HTTP status will be 400 (bad request) if the API call was invalid or 500 if the task failed to launch for some other reason.

If the task was successfully launched (i.e. if a task `id` was returned):

- The text output of the task may be retrieved via the `/api/tasks/id/log` API.
- The task metadata may be retrieved again via the `/api/tasks/id` API.

The server will expire task metadata and logs after 24h has expired since the completion of the task, after which they will no longer be available.

Asynchronous Task API If the caller prefers an asynchronous task interface the request may include the optional argument `async=true`. In this case the server will respond with the task (or API error) metadata without waiting for the task to complete. The HTTP status will be 200 if the task launched successfully, 400 if the API call was invalid, or 500 if the task failed to launch.

If the task was successfully launched the caller may monitor execution of the task by subsequently polling the `/api/tasks/id` API. The `success`, `exitCode`, `error`,

and `ended` fields are only valid once the task is completed, i.e. `running=false`.

Text Task API If the caller prefers to receive the text output of the task instead of the task metadata the request may include the optional argument `text=true` or `live=true` (the latter takes precedence over the former, and `async=true` takes precedence over both).

When `text=true` the server returns the text output of the task as `text/plain` synchronously, i.e. all at once when the task completes.

When `live=true` the server returns the text output of the task as `text/plain` progressively, i.e. line-by-line as it is produced.

In either case

- If the server failed to launch the task it will return API error metadata as `application/json` and an HTTP status code of 400 or 500.
- If the task was successfully launched the first line of output will give the task ID.
- If the task fails an error message will be appended to the text output.

If the task was successfully launched

- For `text=true` the HTTP status will be 200 or 500 depending on whether the task completed successfully.
- For `live=true` the HTTP status will be 200 whether or not the task completed successfully (because the status code may need to be sent before the task completes).

Get Master Task ID: GET /api/tasks/master/id

Returns id of the master task.

Example:

Request

```
GET /api/tasks/master/id HTTP/1.1
Host: SERVER_URL
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

0

Get Task Metadata: GET /api/tasks/*id*

Returns JSON metadata for task with the given id.

Example: get metadata for task 0 (typically task 0 is the master task)

Request

```
GET /api/tasks/0 HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

```
HTTP/1.1 200
status: 200
content-type: application/json; charset=utf-8
```

```
{
  "id": 0,
  "running": true,
  "success": false,
  "exitCode": null,
  "error": null,
  "started": 1537227621242,
  "ended": null
}
```

Get Task Log: GET /api/tasks/*id*/log

Returns the log for task with the given id.

By default the log is returned as an JSON array of strings containing the line-by-line text output of the task. If the task is still running then whatever it has output up to the time of call will be returned.

If the argument `text=true` is specified then the output is `text/plain` lines instead of a JSON array.

If the argument `live=true` is specified then the output is `text/plain` lines starting with the log to-date. If the task is still running then subsequent lines will be sent progressively until the task is complete.

Example: get text log of task 0 (typically task 0 is the master task)

Request

```
GET /api/tasks/0/log?text=true HTTP/1.1
Host: https://landform.hi.jpl.nasa.gov
x-landform-token: API_TOKEN
```

Response

HTTP/1.1 200
status: 200
content-type: text/plain; charset=utf-8

2018-09-17 17:17:55,414 OPS.Pipeline.TileServer.PipelineStateMachine: DefineTiles project:tes
2018-09-17 17:18:02,225 OPS.Pipeline.TileServer.PipelineStateMachine: ChunkInput project:tes
2018-09-17 17:18:02,263 OPS.Pipeline.TileServer.PipelineStateMachine: Build Leaves
2018-09-17 17:18:13,798 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:16,570 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:25,463 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:26,130 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:27,726 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:29,523 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:30,705 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:31,080 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:32,427 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:34,871 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:38,479 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:40,314 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:40,531 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:42,662 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:44,208 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:44,427 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:47,184 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:47,545 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:52,385 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:55,313 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes
2018-09-17 17:18:56,468 OPS.Pipeline.TileServer.PipelineStateMachine: TileCompleted project:tes

Test Procedures

Authentication Test

1. Note this will only work on <https://landform.hi.jpl.nasa.gov> because that is the domain SSO is configured to use.
2. Click **API Token** and confirm the response is **not authenticated**.
3. Click **Login (SSO)**.
4. Enter credentials for a JPL user identity that is a member of the LDAP group configured when the Landform master server was deployed. For test and production deployments managed by the Landform team, this is **landform**.
5. Click **API Token** and confirm the response is a web token - copy the token ID for later.
6. Click **Logout**.
7. Click **API Token** and confirm the response is **not authenticated**.

Note: SSO will only work in a deployment where the Landform master server DNS name matches a configuration registered with the JPL SSO service. If you are running the server locally within the JPL IP address space, e.g. for testing, then you can use the LDAP login instead of SSO.

REST API Tests

Three methods are supported for running REST API tests:

1. Using `curl` (<https://curl.haxx.se/windows>) from the Windows 10 command prompt (`cmd`)
2. Using `curl` from any other compatible command prompt.
3. Using Postman (<https://www.getpostman.com>). This is the recommended method as it is mostly automated.

Details are given below for `curl` on the Windows 10 command prompt. To use another compatible command prompt you may need to modify the syntax slightly. For example, in `bash` style command prompts variable substitution syntax is `$VARIABLE` instead of `%VARIABLE%`.

To use Postman

1. download and import this Postman collection
2. define variables
 1. ellipsis menu for collection -> Edit -> Variables
 2. set the “current value” for each variable following the Setup procedure below
 3. Update
3. Follow the test sequence below, but using the pre-defined requests in the Postman collection instead of `curl`. For example, to create the project, click the “create project” request in the collection and then click “Send”.

Setup

1. Login and get an API token using the authentication test procedure. Copy the token to the system clipboard, and then paste it on the windows command line to set a temporary environment variable by running a command like this:

```
set API_TOKEN=(pasted token)
```

2. Set the server URL. For example, to test against the production or test server managed by the Landform team:

```
set SERVER_URL=https://landform[-dev].hi.jpl.nasa.gov
```

Otherwise substitute the end-user DNS name of your Landform master server.

3. Determine a unique name for a test project. For example, each time you run this procedure use a project name like `testN` where `N` is an integer that you increment.

```
set PROJECT_NAME=testN
```

4. Select a mesh and texture file for the test project. Several test datasets are available at <https://landlords-dev.s3.amazonaws.com/landformweb-test-data/shared.zip>.

```
set MESH_FILE=shared/stick/stick.ply set TEXTURE_FILE=shared/stick/stick.jpg
```

Perform the remaining steps of this procedure in the same command window where these variables were set.

Test Sequence

1. Create project:

```
curl -sS --request POST
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON object with `success=true`.

2. List projects:

```
curl -sS --request GET
      --url "%SERVER_URL%/api/projects"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON array of strings containing `%PROJECT_NAME%`.

3. Upload input files:

```
curl -sS --request POST
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%/upload"
      --header "x-landform-token: %API_TOKEN%"
      --form "mesh=@%MESH_FILE%"
      --form "texture=@%TEXTURE_FILE%"
```

If using Postman select the “upload data” request, then click Body, then Choose Files.

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON object with `success=true`.

4. Run project:

```
curl -sS --request POST
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%/run"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON object with `success=true`.

5. Get project metadata:

```
curl -sS --request GET
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON object.

6. Poll project metadata about every 10 seconds until `Project.FinishedRunning` is `true`.

7. Get project result URL:

```
curl -sS --request GET
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%/result?redirect=false"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid URL.

8. Get project viewer URL:

```
curl -sS --request GET
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%/view?redirect=false"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid URL.

9. Load the project viewer URL in a Chrome browser to visually inspect the result.

10. Delete project:

```
curl -sS --request DELETE
      --url "%SERVER_URL%/api/projects/%PROJECT_NAME%"
      --header "x-landform-token: %API_TOKEN%"
```

Validation: check that the response code is HTTP 200 (ok), the content type is `application/json`, and the response body is a valid JSON object with `success=true`.